St. Francis Institute of Technology

(An Autonomous Institution)

AICTE Approved | Affiliated to University of Mumbai
A+ Grade by NAAC: CMPN, EXTC, INFT NBA Accredited: ISO 9001:2015 Certified

Department of Information Technology

A.Y. 2025-2026
Class: BE-IT A/B, Semester: VII

Subject: Secure Application Development Lab

Student Name:Vishal Rajesh Mahajan Student Roll No: 1

Experiment – 4: Study of different SAST (Static Application Security Testing) tools

Aim: To study and exercise on different SAST (Static Application Security Testing) tools

Objective: After performing the experiment, the students will be able to –

- To get familiar with the features provided by the open source SAST tools on GitHub

Lab objective mapped: ITL703.2: To understand the methodologies and standards for developing secure code

Prerequisite: Basic knowledge Information Security, software enginerring

Requirements: Personal Computer, Windows operating system browser, Internet Connection etc.

Pre-Experiment Theory:

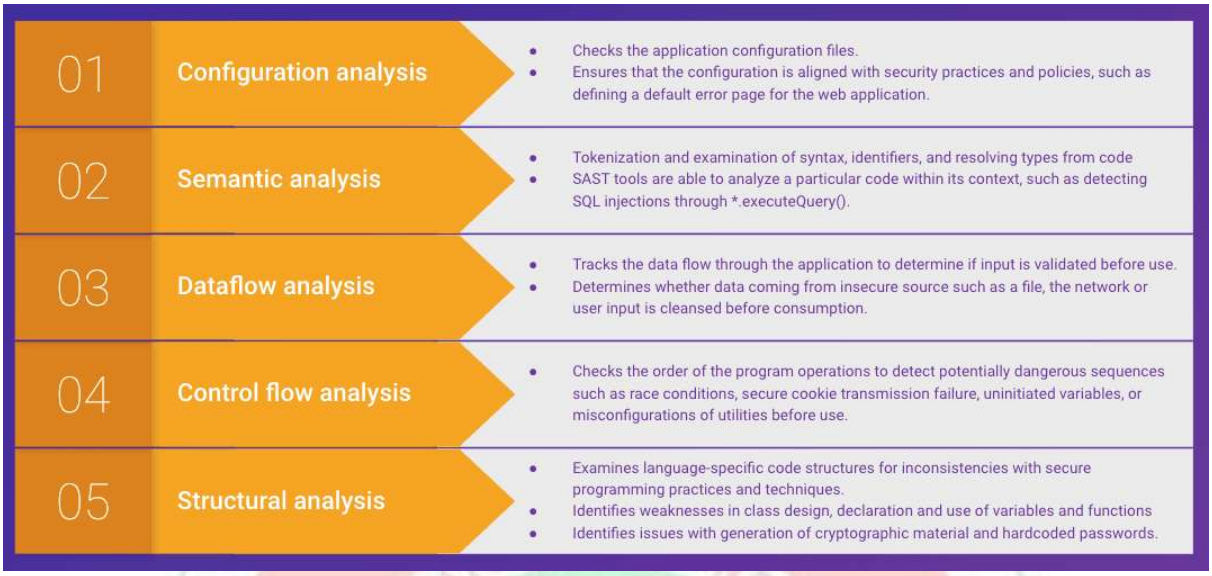
What is Static Application Security Testing (SAST)?

SAST is a vulnerability scanning technique that focuses on source code, byte code, or assembly code. The scanner can run early in your CI pipeline or even as an IDE plugin while coding. SAST tools monitor your code, ensure protection from security issues such as saving a password in clear text or sending data over an unencrypted connection.

How do SAST tools work?

SAST is a technique used to evaluate source code without actually executing it. It involves examining the program's structure and syntax to identify potential issues and errors, such as coding mistakes, security vulnerabilities, and performance bottlenecks. The process involves parsing the source code, building an abstract syntax tree, and applying various analysis

techniques to detect issues. By providing early feedback on potential issues in the code, SAST can help improve software quality and reduce the likelihood of errors and security vulnerabilities.



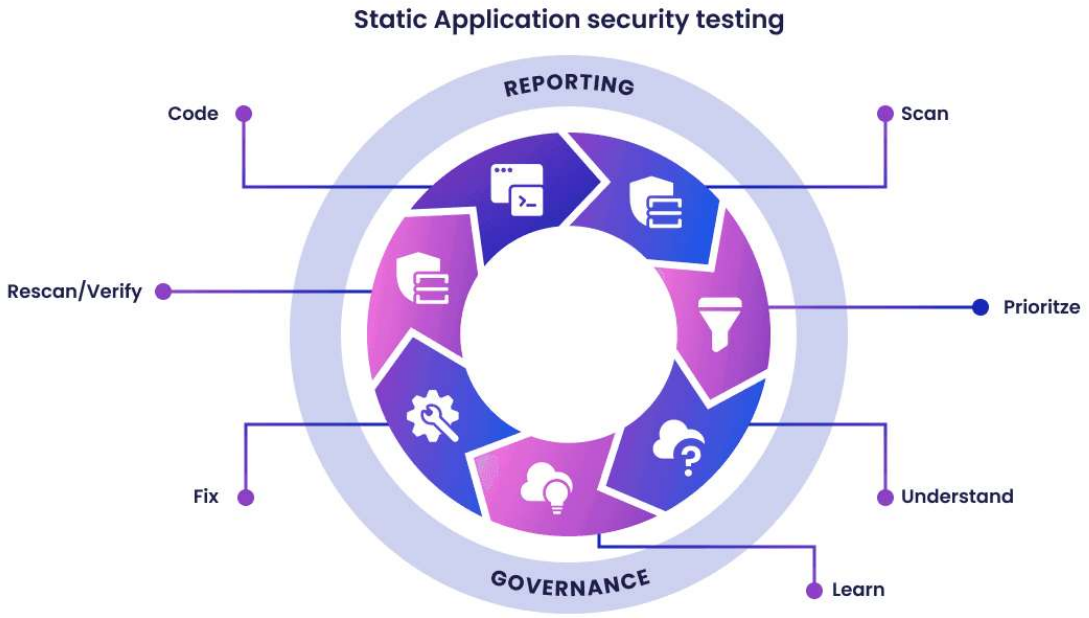
What vulnerabilities can SAST tools find?

SAST tools detect a range of security incidents and vulnerabilities in source code, including:

- Dataflow issues
- Semantic errors
- Misconfigured settings
- Control flow problems
- Structural flaws
- Memory issues

7 stages of a security application testing scan

The seven stages of a SAST scan are:



1. **Scan** your code as it's being written.
2. **Prioritize** and triage based on the severity and impact of the vulnerabilities found
3. **Understand** the nature of the vulnerabilities found by reviewing scan data and assessing the associated risk level.
4. **Learn** from the scan findings to prevent similar vulnerabilities in the future. This includes improving code quality, adopting secure coding practices, and implementing developer security training.
5. **Fix** vulnerabilities found in the scan by patching the code or implementing other remediation measures.
6. **Rescan** to verify that the fix has worked.
7. **Continue** coding while integrating security into the development process to prevent vulnerabilities from being introduced in future code.

Procedure:

Select any **five** open source SAST tools from GitHub and discuss them in detail.

RUFF (PYTHON)

Ruff is an extremely fast and efficient linter and code formatter for Python, written in Rust. It aims to replace and consolidate many existing Python tools like Flake8, isort, and Black, offering a unified solution for code quality and consistency.

Ruff Key Features

- Ruff is a Python linter and code formatter written in Rust, designed to be over 10–100x faster than Flake8 or Black.
- It supports drop-in parity for Flake8, isort, Black, and can replace dozens of Python code hygiene plugins, covering style, import order, documentation, and upgrades.
- Over 800 linting rules are natively implemented, with hierarchical, monorepo-friendly configuration, real-time editor integration (VS Code, PyCharm), and built-in caching to avoid redundant analysis.

Ruff Advantages

- Large codebases can be linted in seconds, not minutes, with minimal CPU and RAM use, making Ruff ideal for continuous integration (CI) and frequent code reviews.
- Automatic error correction—such as fixing unused imports—with simple commands streamlines team workflows and eliminates many manual fixes.
- Ruff is actively maintained, regularly supporting the latest Python syntax and major open-source libraries like Apache Airflow, Superset, FastAPI, Pandas, SciPy, and HuggingFace.

Developer Experience

- Developers see instant feedback in their IDEs and commit hooks, drastically reducing waiting time and increasing code hygiene before pull requests are made.
- Ruff combines the benefits of style checking, error detection, and auto-fixes under a unified system, replacing what previously required several separate tools.
- A strong and growing Rust-backed community provides fast support and frequent updates, while the single-command install (pip install ruff) ensures low friction onboarding even for large teams

RETIRE.JS(JAVASCRIPT)

Retire.js is an open-source security tool designed to detect JavaScript libraries and Node.js modules with known vulnerabilities. It helps developers identify and fix insecure or outdated dependencies, reducing security risks in web applications and server-side JavaScript projects.

Features

- Detects vulnerable JavaScript libraries and Node.js modules using a comprehensive, regularly updated vulnerability database.
- Operates via command line, Grunt and Gulp plugins, browser extensions, and integrates with security tools like Burp Suite and OWASP ZAP.
- Generates Software Bill of Materials (SBOM) in CycloneDX format for effective dependency management.

Advantages

- Early identification of security risks in dependencies reduces potential exploitation.
- Flexible integration into various development and CI/CD workflows ensures continuous security scanning.
- Helps maintain compliance with OWASP top security practices by managing third-party component vulnerabilities.

Developer Experience

- Easy to install and use with clear, actionable reports guiding remediation.
- Provides real-time feedback in development environments via plugins and extensions.
- Enhances developer productivity by embedding security checks seamlessly into the build and testing processes.

LINTR (R)

lintr is an open-source static code analysis tool for the R programming language. It helps developers detect style issues, potential bugs, and coding errors by analyzing R scripts and enforcing style guidelines to improve code readability and quality across projects.

Features

- Detects syntax errors, style violations, and semantic problems in R code.
- Provides customizable linters for enforcing coding standards.
- Supports integration with editors like RStudio, Emacs, and Vim, offering real-time linting feedback.
- Easily integrates with CI pipelines and supports GitHub Actions for automated code quality checks.

Advantages

- Promotes consistent coding style across R projects, improving maintainability.
- Helps catch bugs early by identifying problematic code patterns before runtime.
- Flexible and extensible with the ability to customize or add new linters.
- Integrates smoothly into development workflows, supporting continuous code quality.

Developer Experience

- Simple to install and configure through R packages with user-friendly documentation.
- Provides real-time linting in popular R development environments for immediate feedback.
- Detailed, actionable warnings and suggestions guide developers to improve code quality.
- Automates repetitive code review tasks, freeing developers to focus on logic and feature development.

SQLINT (SQL)

sqlint is an open-source static analysis tool designed to lint SQL code by checking for syntax errors and best practices adherence. It primarily supports ANSI SQL and uses the PostgreSQL parser to validate SQL scripts, helping developers maintain clean and error-free SQL codebases.

Features

- Detects SQL syntax errors and potential issues based on ANSI SQL standards.
- Supports multiple text editor integrations including Emacs, Vim, and SublimeText.
- Provides command-line interface for seamless integration with CI/CD pipelines and pre-commit hooks.
- Lightweight tool with simple configuration, focused on SQL code quality enforcement.

Advantages

- Helps catch SQL errors early before runtime, reducing debugging and production issues.
- Ensures consistent SQL style and adherence to standards across teams.
- Enables integration in automated testing and development workflows for continuous quality.
- Supports common SQL dialects via PostgreSQL parser, increasing its versatility.

Developer Experience

- Easy to install and configure, ideal for developers working primarily with SQL.
- Real-time feedback in editors enhances coding efficiency and accuracy.
- Command-line interface and editor plugins enable smooth workflow integration.
- Lightweight tool that does not add significant overhead to development processes.

Zod (Typescript)

Zod is an open-source TypeScript schema validation library that combines static type inference with runtime validation. It helps developers define and enforce schemas for data structures, ensuring correctness and type safety in TypeScript applications.

Features

- Provides comprehensive schema declaration for primitives, objects, arrays, unions, and intersections.
- Enables static type inference from schemas, reducing boilerplate and errors.
- Supports transformations, refinements, and custom validations at runtime.
- Integrates easily with TypeScript projects and popular frameworks like React and Node.js.

Advantages

- Ensures data validity both at compile time and runtime, preventing common bugs.
- Simplifies complex data validation logic with expressive and reusable schemas.
- Enhances maintainability by centralizing data shape definitions and validation rules.
- Small footprint and zero dependencies make it lightweight and performant.

Developer Experience

- Seamless integration with TypeScript tooling provides real-time feedback on schema correctness.
- Clear error messages and detailed validation failures improve debugging.
- Reduces manual coding effort by auto-inferring types and generating validations.
- Widely adopted with strong community support and extensive documentation.Here is a detailed discussion of 5 open source SAST tools from GitHub, based on your input technologies:

Post-Experimental Exercise

Questions:

- How to find the right SAST tool to secure the software development lifecycle (SDLC)?
- Compare SAST with DAST

Conclusion:

- Write what was performed in the experiment.
- Write the significance of the topic studied in the experiment.

References

- https://snyk.io/learn/application-security/static-application-security-testing/#how
- https://github.com/analysis-tools-dev/static-analysis